

Practical 7

```
In [1]: # Import necessary libraries
import tarfile
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.neural_network import MLPRegressor

# Define a filter function that allows only regular files to be extracted
def tar_safe_filter(tarinfo, path=None):
    if tarinfo.isreg(): # Check if it's a regular file
        return tarinfo # Include this file in extraction
    else:
        return None # Skip non-regular files
    # Open and extract the .tgz file using the filter argument
    with tarfile.open("cal_housing.tgz", "r:gz") as tar:
        tar.extractall(path=".", filter=tar_safe_filter)

# Load dataset from the extracted file
california_data = pd.read_csv("cal_housing.data", header=None, delimiter=",")

# Assign column names
california_data.columns = [
    "MedInc", "HouseAge", "AveRooms", "AveBedrms", "Population",
    "AveOccup", "Latitude", "Longitude", "MedHouseVal"
]

# Separate features and target variable
X_reg = california_data.drop('MedHouseVal', axis=1)
y_reg = california_data['MedHouseVal']

# Split the data for training/testing
```

```

X_train, X_test, y_train, y_test = train_test_split(X_reg, y_reg, test_size=0.3, random_state=42)
print("Dataset loaded successfully.000")

# Define and Train ANN model
model = MLPRegressor(hidden_layer_sizes=(64, 32), max_iter=500, random_state=42)
model.fit(X_train, y_train)
print("Dataset loaded successfully.")

# Evaluate on test data
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("Dataset loaded successfully.5e654")
print(f"MSE: {mse}")
print(f"R² Score: {r2}")
cv_scores = cross_val_score(model, X_reg, y_reg, cv=5, scoring='r2')
print(f"Cross-Validation R² Scores: {cv_scores}")

```

Dataset loaded successfully.000
 Dataset loaded successfully.
 Dataset loaded successfully.5e654
 MSE: 4551438423.780903
 R² Score: 0.6532343357216204

C:\Users\Dell\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\normalization_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500) reached and the optimization hasn't converged yet.

warnings.warn(
 C:\Users\Dell\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\normalization_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500) reached and the optimization hasn't converged yet.
 warnings.warn(

Cross-Validation R² Scores: [0.63140471 0.65227663 0.63824993 0.52074221 0.56007787]

In [2]:

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_predict
from sklearn.metrics import classification_report, roc_auc_score, confusion_matrix
from sklearn.neural_network import MLPClassifier

```

```
from sklearn.preprocessing import LabelBinarizer
from sklearn.metrics import classification_report, roc_auc_score

# Load the Iris dataset
iris = load_iris()
X_clf, y_clf = iris.data, iris.target

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X_clf, y_clf, test_size=0.3, random_state=42)

# Define and Train ANN model
model = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=500, random_state=42)
model.fit(X_train, y_train)

# Evaluate on test data
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))

# Compute confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

# Compute AUC
lb = LabelBinarizer()
y_test_bin = lb.fit_transform(y_test)
y_pred_prob = model.predict_proba(X_test)
auc = roc_auc_score(y_test_bin, y_pred_prob, multi_class='ovr')
print(f"AUC Score: {auc}")
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Confusion Matrix:

```
[[19  0  0]
```

```
 [ 0 13  0]
```

```
 [ 0  0 13]]
```

AUC Score: 1.0